# CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

## Lecture 15: Single-Cycle Datapath II

Instructor: Ariana Abel

Slides Credit: Justin Yokota

# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing

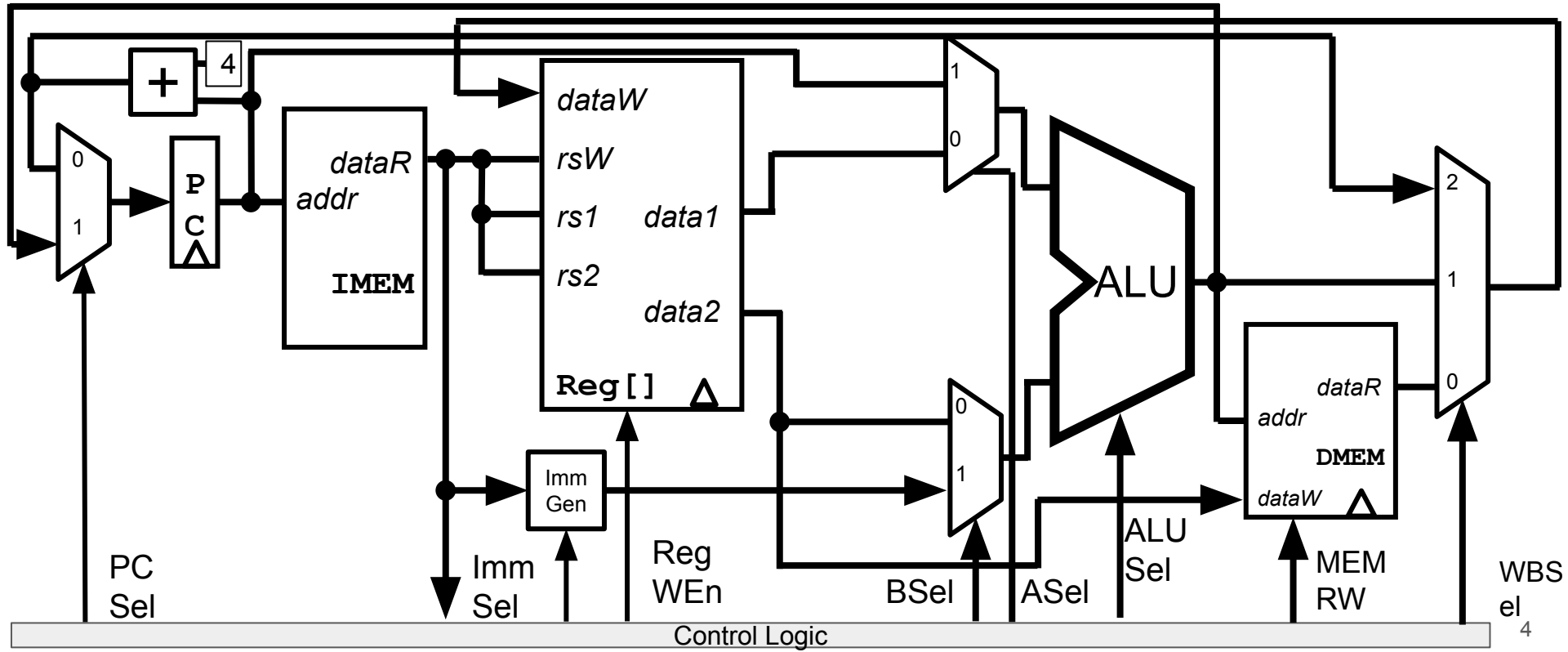
# Agenda



- **Implementing Branches**
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing

# Datapath so far: R, I, S, J types



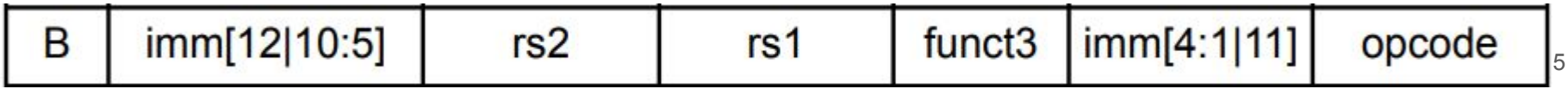


# Implementing branch instructions



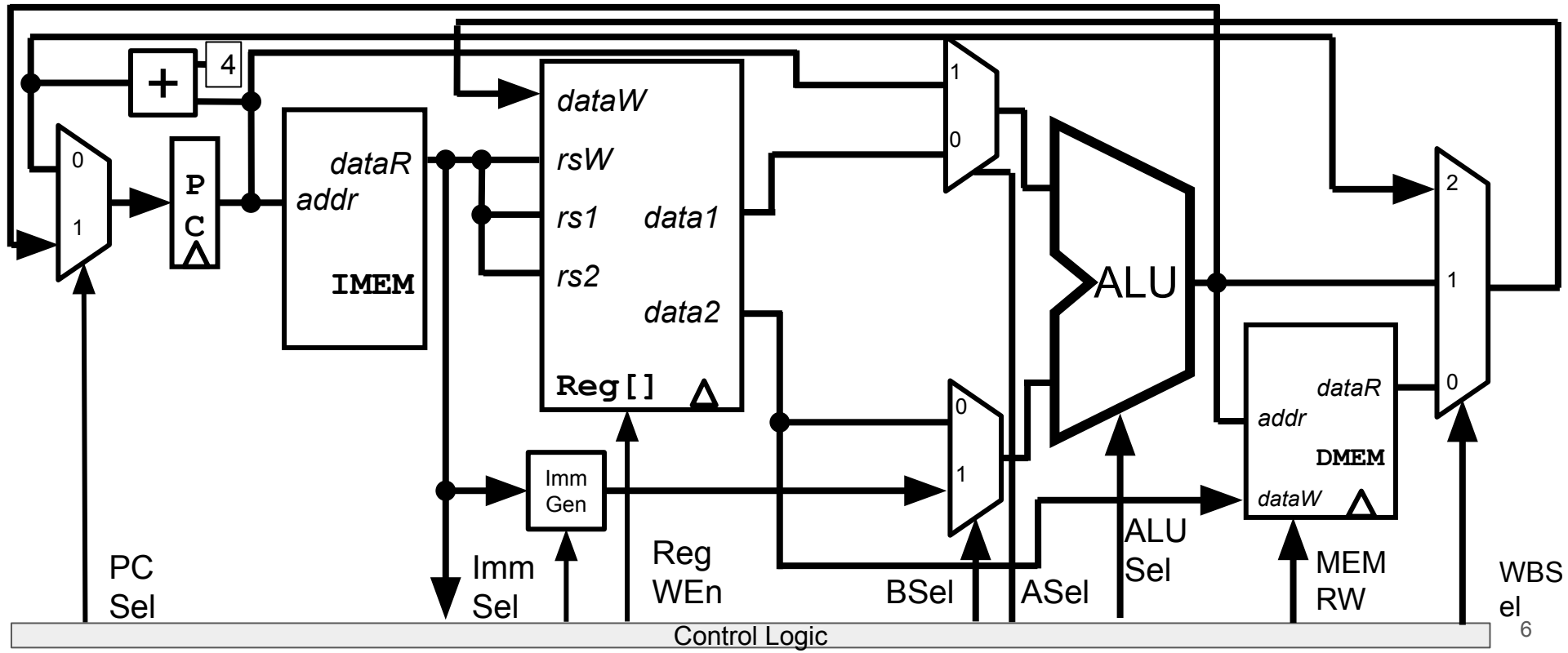
- Let's add `beq`
- `beq` compares rs1 and rs2:
  - Need new component for comparisons
- Then computes one thing:
  - New PC if branch = PC+offset
    - This uses the ALU, so can't use ALU for comparisons - need new block!
- Then sets one state element:
  - PC+4 if branch not taken, PC+offset if branch is taken
  - Can use PCSel
- Main issues:
  - Control logic (PCSel) needs to be conditioned on Branch result
  - New immediate type
  - New Branch Comparator

| | | | | |
|---|---|---|---|---|
| `beq rs1 rs2 label` | Branch if EQual | `if (rs1 == rs2) PC = PC + offset` | B | `110 0011` | `000` |



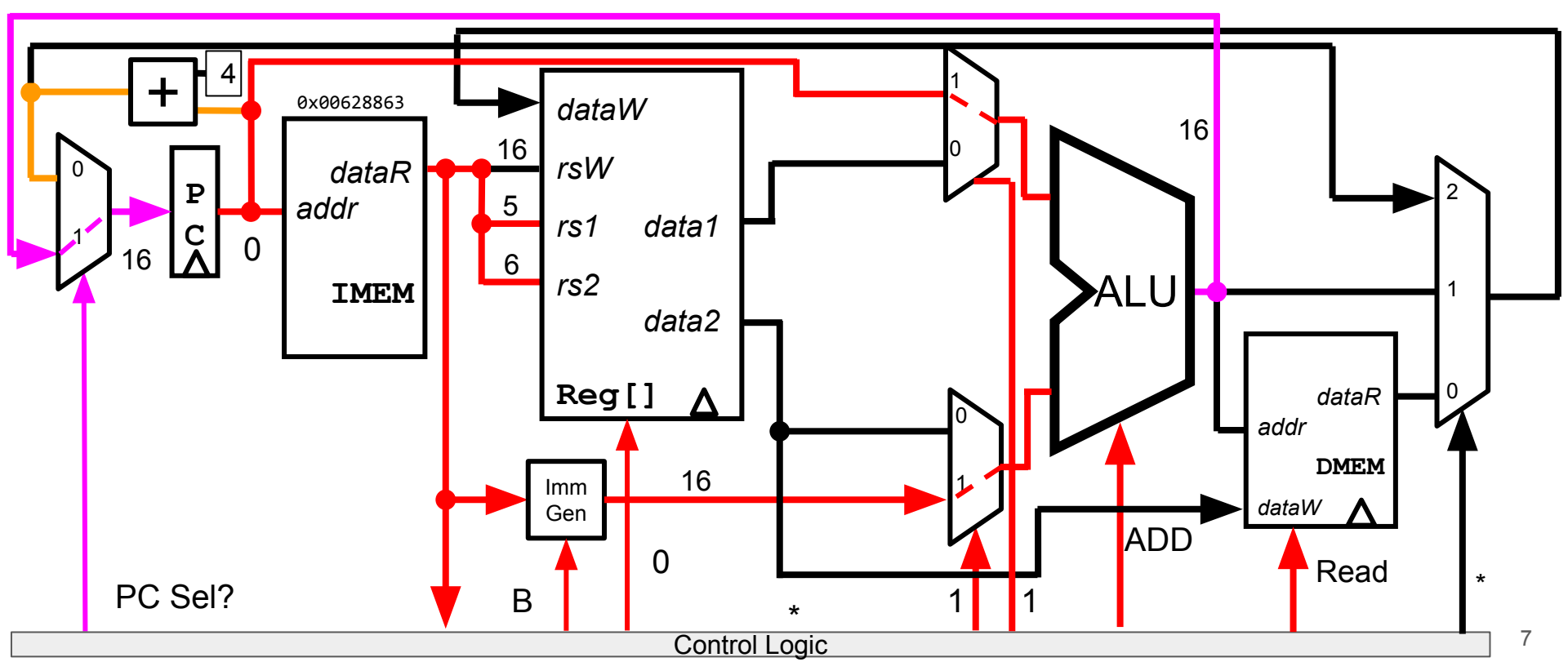
# Datapath so far





# Datapath running beq x5 x6 16





# The Branch Comparator



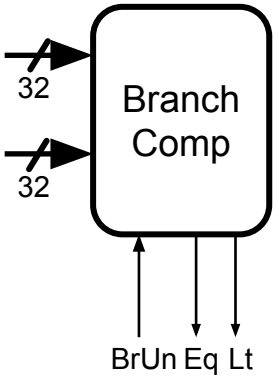
The Branch Comparator will handle all our branch instructions:

Input:

- Two data busses A and B (corresponding to rs1 and rs2)
- BrUn control bit: Do unsigned comparison?
  - Only affects BrLt, since == is the same regardless of signed vs unsigned

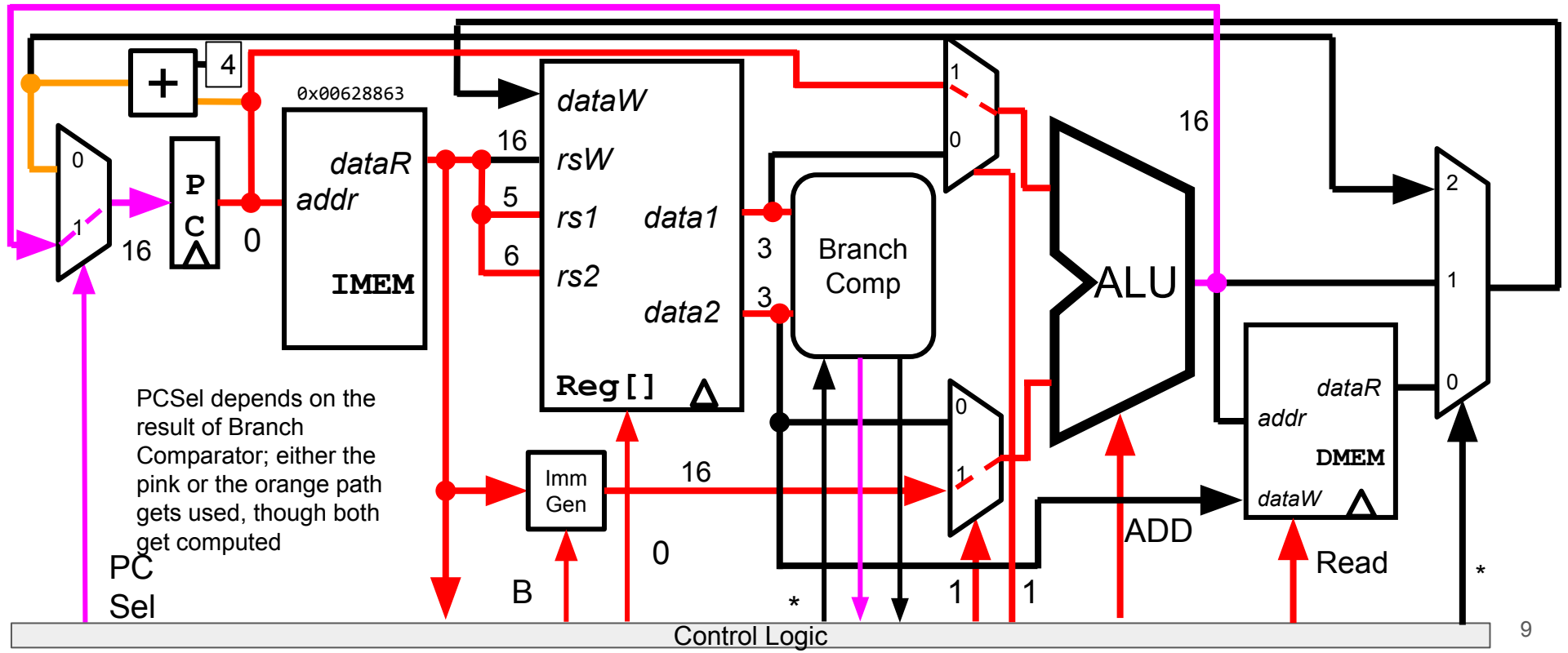
Output:

- Two bits:
  - BrEq that's 1 if A==B
  - BrLt that's 1 if A < B
  - Note: A>=B is the same as !(A<B), so no need for BrGe
- Can decide PCSel based on these outputs, so send to Control Logic



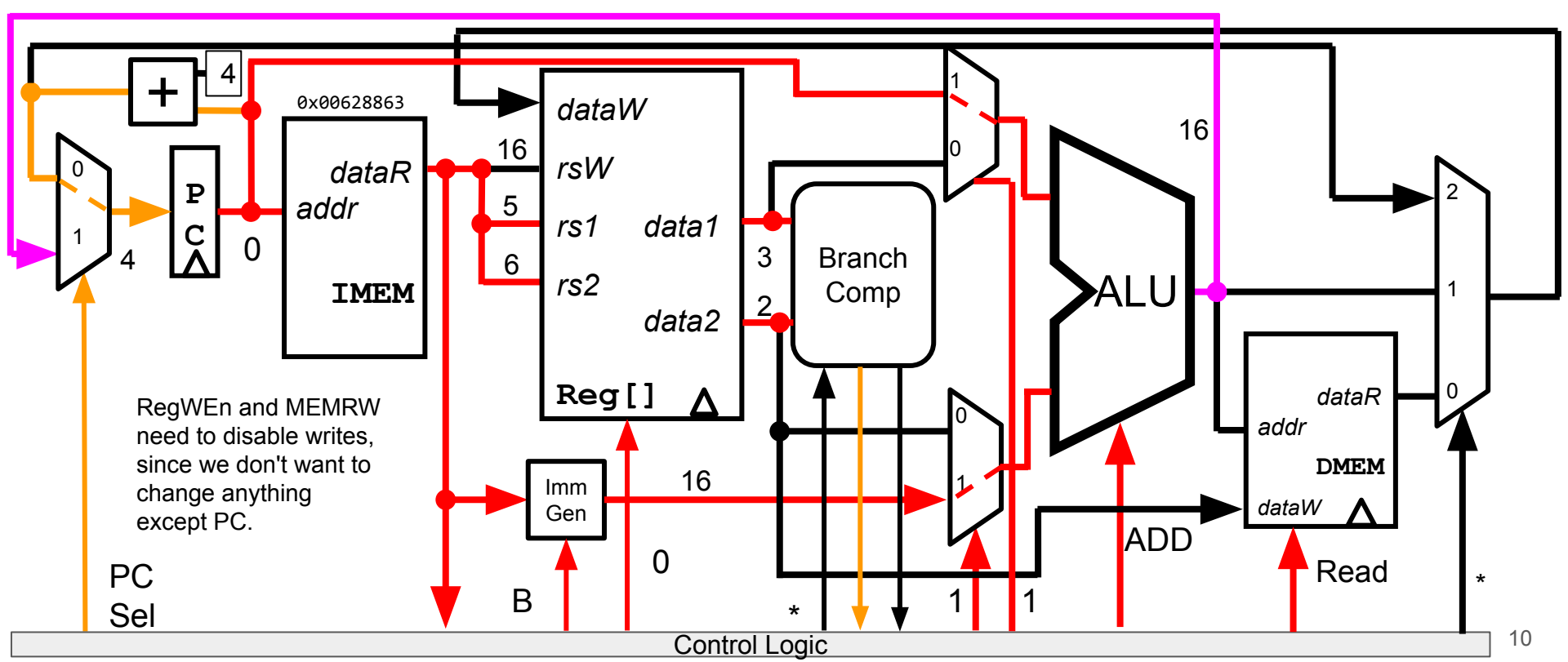
# Datapath running `beq x5 x6 16` (equality holds)





# Datapath running `beq x5 x6 16` (equality does not hold)





# Agenda



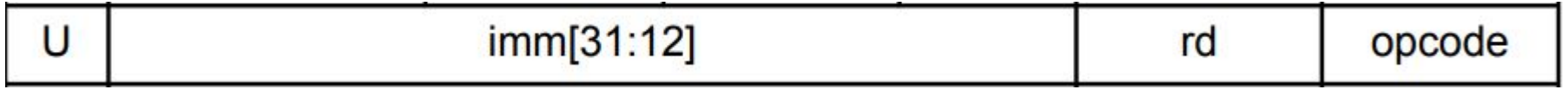
- Implementing Branches
- **Implementing U-types**
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing

# Implementing `lui`



- Let's add `lui`
- `lui` computes a new immediate type
- And affects two state elements:
  - rd = imm
  - PC = PC+4

| | | | | | |
|---|---|---|---|---|---|
| `auipc rd immu` | Add Upper Immediate to PC | `imm = immu << 12`<br>`rd = PC + imm` | U | `001 0111` | |
| `lui rd immu` | Load Upper Immediate | `imm = immu << 12`<br>`rd = imm` | U | `011 0111` | |

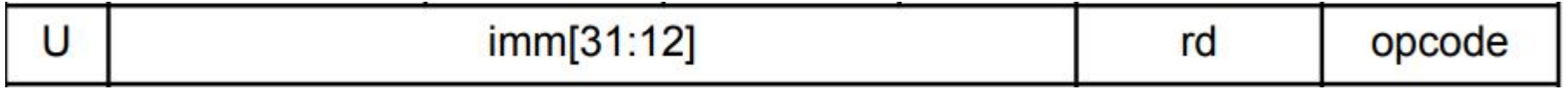


# Implementing `auipc`



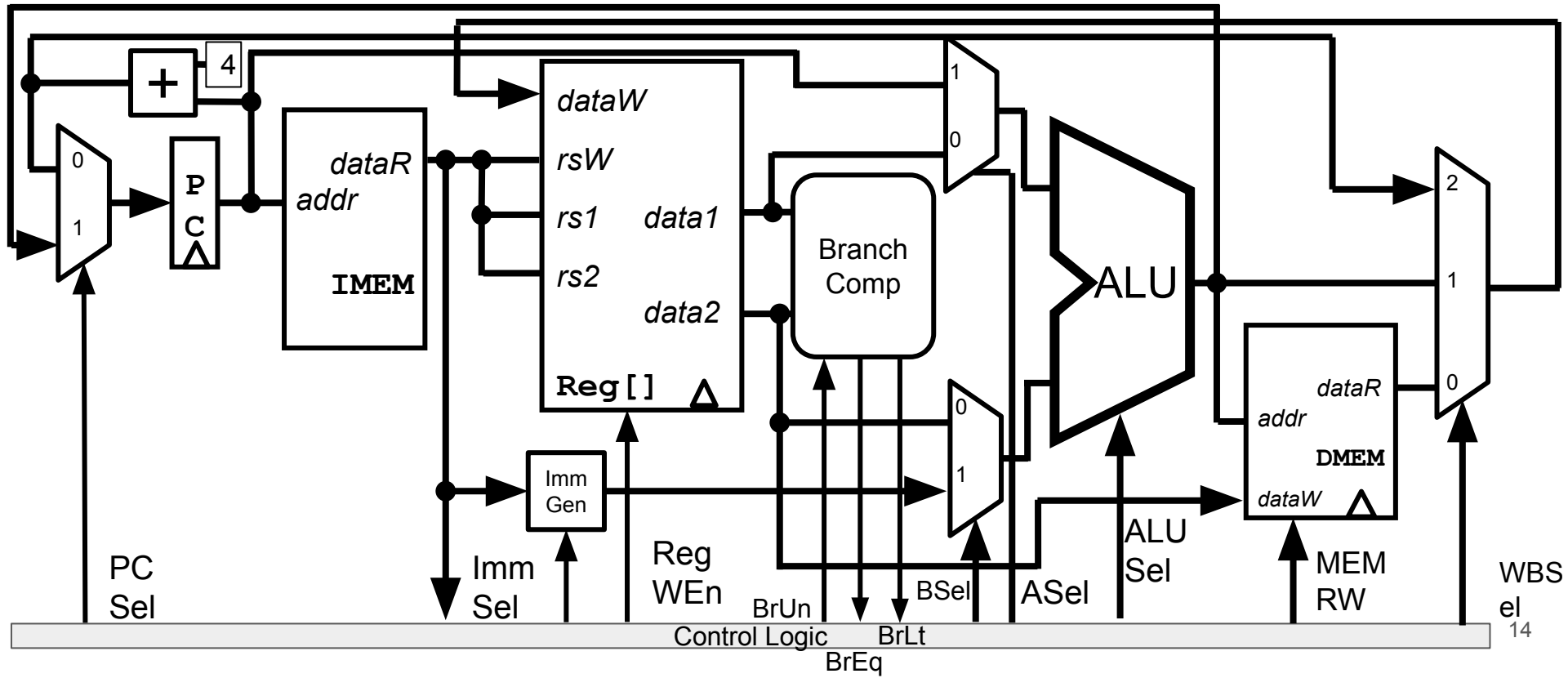
- `auipc` does the same thing as `lui`, but it also adds PC to the immediate value

| | | | | | |
|---|---|---|---|---|---|
| `auipc rd immu` | Add Upper Immediate to PC | `imm = immu << 12`<br>`rd = PC + imm` | U | `001 0111` | |
| `lui rd immu` | Load Upper Immediate | `imm = immu << 12`<br>`rd = imm` | U | `011 0111` | |



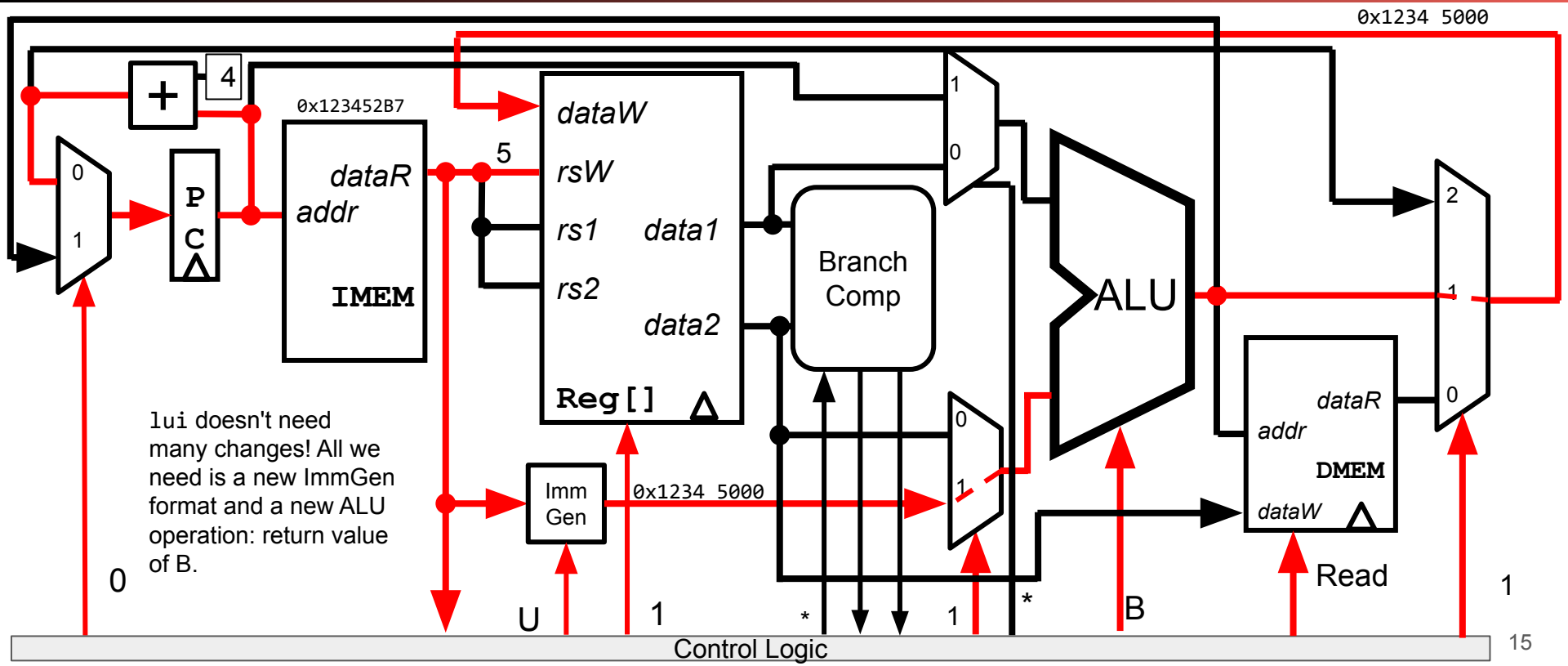
# Datapath So Far





# Datapath running `lui x5 0x12345`

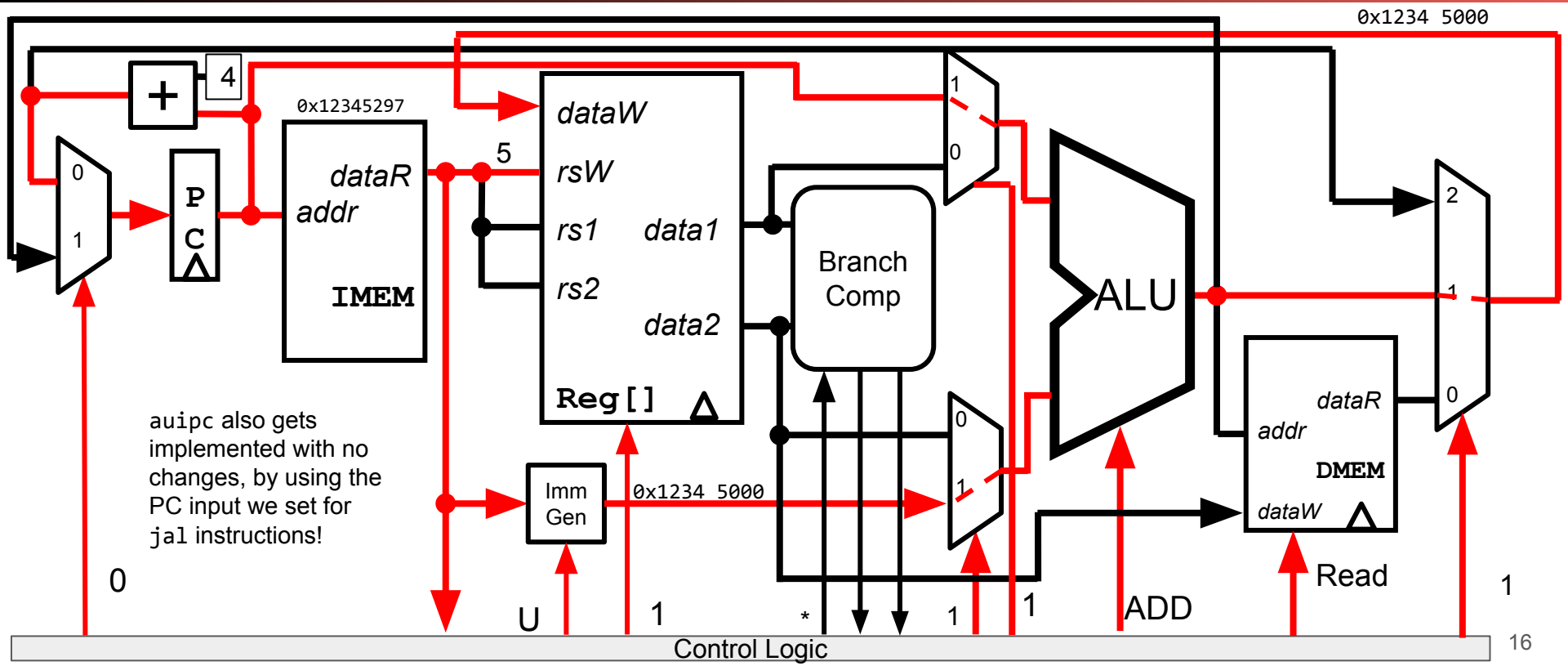





`lui` doesn't need many changes! All we need is a new ImmGen format and a new ALU operation: return value of B.

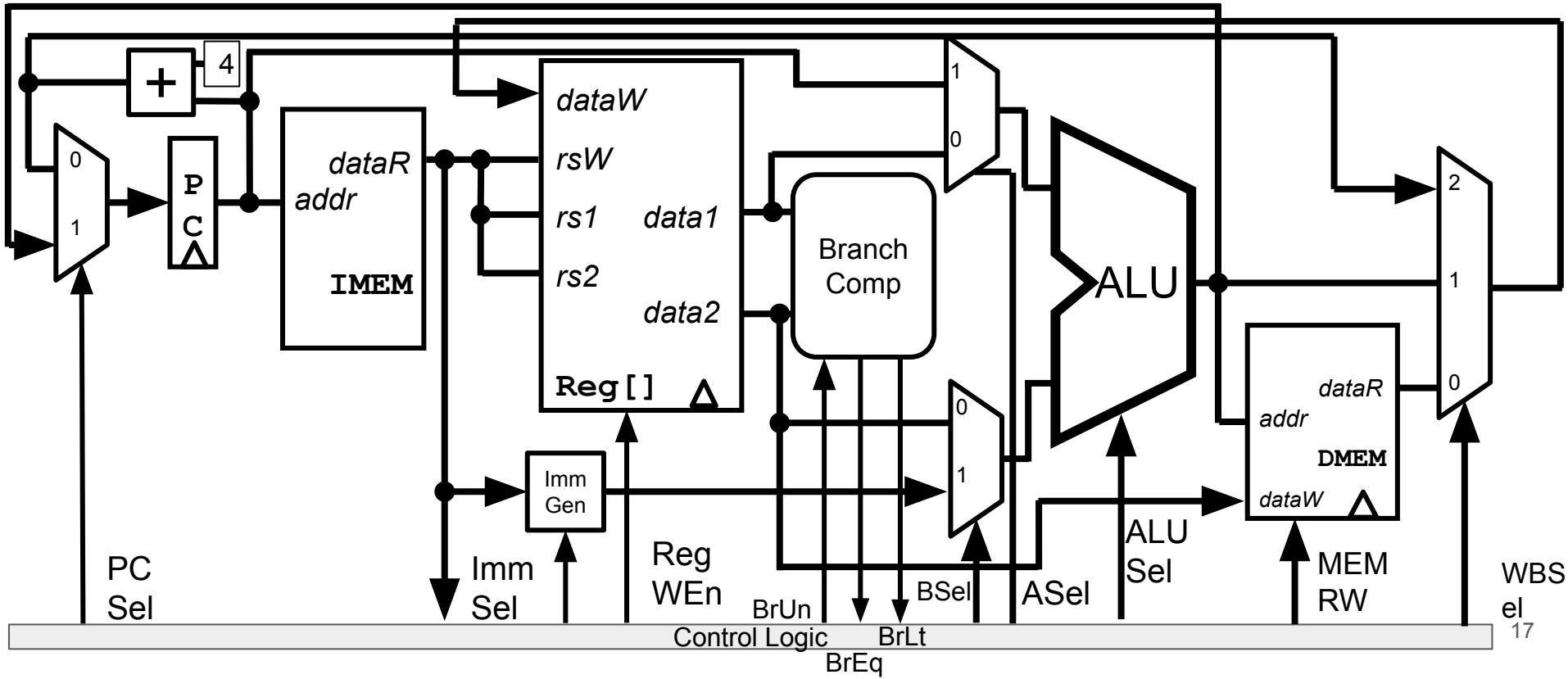
# Datapath running `auipc x5 0x12345`





# Complete RISC-V Datapath!!!





# Agenda



- Implementing Branches
- Implementing U-types
- **Implementing ImmGen**
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing

# What's Left?



With the datapath we have, we've reduced the problem of making a CPU to a few small subcircuits:

- **Multiplexers**: Discussed in previous lecture, and part of Lab 5
- **ALU/Regfile**: Implemented as part of Project 3A
- **Branch Comparator**: Implemented as part of Project 3B
- **IMEM/DMEM**: Exact implementation out of scope
- **Control Logic**: The big one
  - Later in lecture
- **ImmGen**: Let's finish this off right now
  - At the same time, we'll notice some clever patterns in how the bits are stored across instruction formats

# How to make the ImmGen



Option 1:

- Make a circuit for each instruction format, then mux the results with ImmSel
- Ugly, but it works

Option 2:

- Look for patterns in the instruction format
- Takes time, but gives much greater insight into why instruction formats were designed this way

# Instruction Formats



| 31 25 | 24 20 | 19 15 | 14 12 | 11 7 | 6 0 |
|---|---|---|---|---|---|
| R | funct7 | rs2 | rs1 | funct3 | rd | opcode |
| I | imm[11:0] | | rs1 | funct3 | rd | opcode |
| I* | funct7 | imm[4:0] | rs1 | funct3 | rd | opcode |
| S | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode |
| B | imm[12\|10:5] | rs2 | rs1 | funct3 | imm[4:1\|11] | opcode |
| U | imm[31:12] | | | | rd | opcode |
| J | imm[20\|10:1\|11\|19:12] | | | | rd | opcode |

# Tracking all Immediate Bits in each format



| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | | | | | | | | | | | | | | | | | | |
| S | 11 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 0 | | | | | | | |
| B | 12 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 11 | | | | | | | |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |
| J | 20 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |

# Tracking all Immediate Bits in each format



| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

Bit 0 of imm: Bit 20 of inst if I type, Bit 7 of inst if S type, always 0 if B/U/J type

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | | | | | | | | | | | | | | | | | | |
| S | 11 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 0 | | | | | | | |
| B | 12 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 11 | | | | | | | |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |
| J | 20 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |

# Tracking all Immediate Bits in each format



| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

Bits 4-1 of imm: Bits 24-21 of inst if I/J type, Bits 11-8 of inst if S/B type, always 0 if U type. **Only found in two parts of the instruction, and these 4 bits are always together.**

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | | | | | | | | | | | | | | | | | | |
| S | 11 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 0 | | | | | | | |
| B | 12 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 11 | | | | | | | |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |
| J | 20 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |

# Tracking all Immediate Bits in each format



| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

Bits 10-5 of imm: Bits 30-25 of inst if I/S/B/J type, always 0 if U type. **Only found in one part of the instruction, even though 4 different instruction formats use it!**

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | | | | | | | | | | | | | | | | | | |
| S | 11 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 0 | | | | | | | |
| B | 12 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 11 | | | | | | | |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |
| J | 20 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |

# Tracking all Immediate Bits in each format



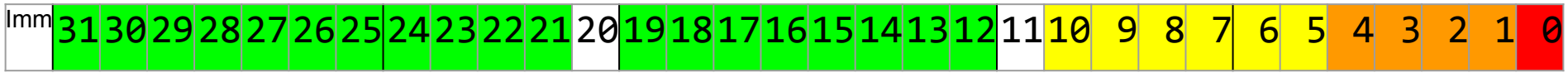
| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

MSB of instruction is ALWAYS the MSB of the immediate. Sign-extending is just copying the MSB, and almost all RISC-V immediates sign-extend. **To sign-extend, we just take the MSB of the instruction.**

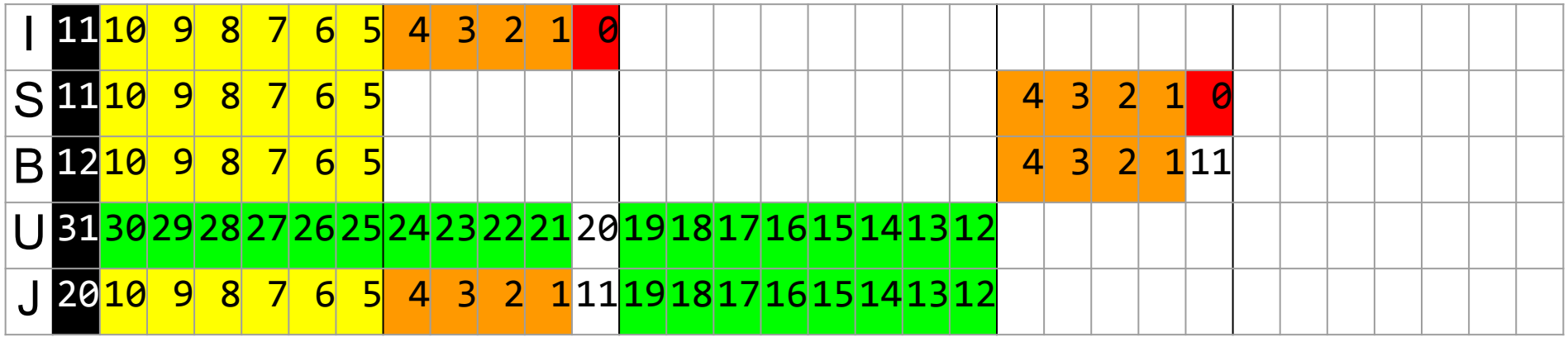
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | | | | | | | | | | | | | | | | | | |
| S | 11 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 0 | | | | | | | |
| B | 12 | 10 | 9 | 8 | 7 | 6 | 5 | | | | | | | | | | | | | | 4 | 3 | 2 | 1 | 11 | | | | | | | |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |
| J | 20 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | | | | | | | | | | | | |

# Tracking all Immediate Bits in each format



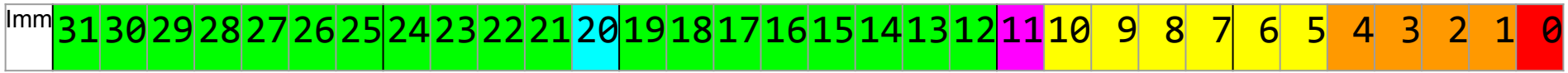



Bits 31-21 and 19-12 of immediate: Always the corresponding bit in the instruction if part of the format, and bit 31 otherwise. **Only two options for these bits as well**

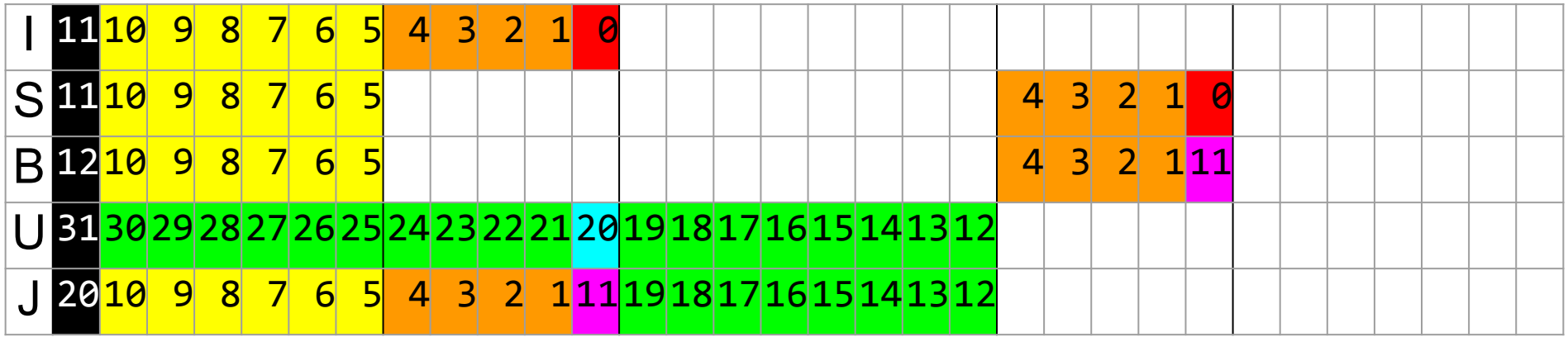


# Tracking all Immediate Bits in each format






Bits 20 and 11 of immediate: Kind of put in random spots where there's space. Bit 20 needs a 2-way mux, and bit 11 needs a 4-way mux



# How to make the ImmGen



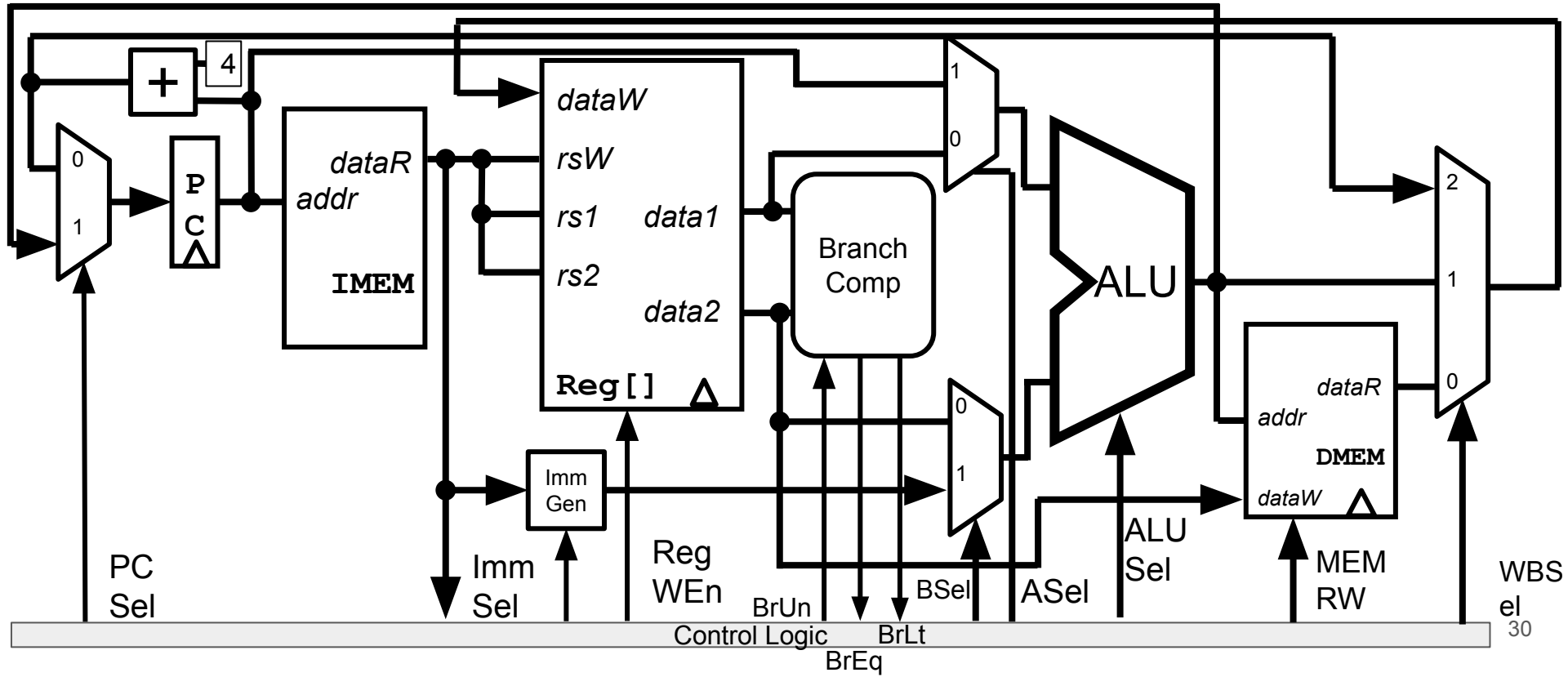
Because of how all instruction formats were chosen, there are a lot of underlying patterns in how immediates are stored.

By looking for these patterns, the ImmGen can be greatly simplified (in terms of total number of logic gates)

- 6-select 32-bit mux = 640 logic gates (counting 2-input OR and ANDs, and 1-input NOTs)
- Using just muxes for each set of bits, you can get this down to 160 logic gates (best Justin got)
  - Con: Makes it harder to extend your CPU. Often not worth it to optimize this too much
- Exact details left for Project 3B

# Complete RISC-V Datapath!





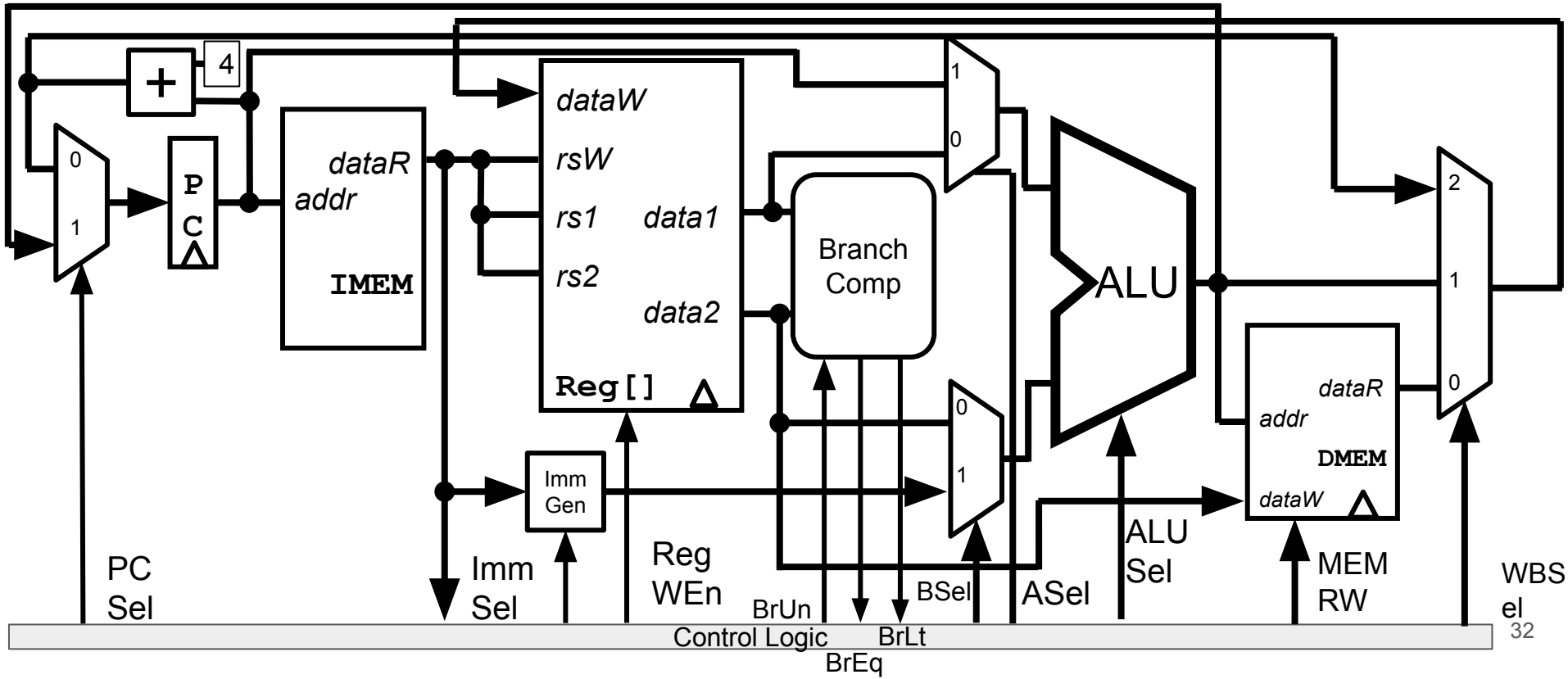
# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- **Datapath and Control**
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing

# Complete RISC-V Datapath





# [Practice] RV32I Datapath and Control



For each instruction below:

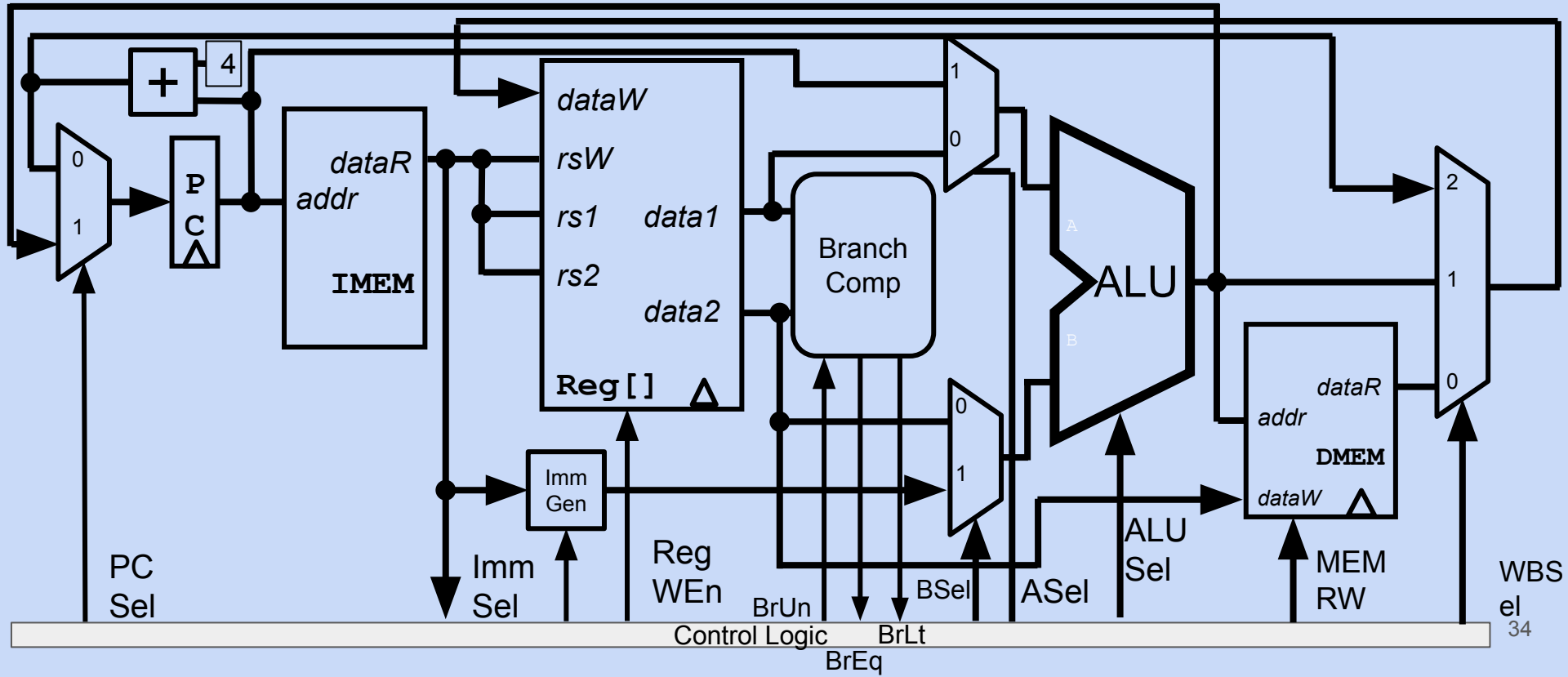
a. Which wires should hold relevant data, i.e., what is the “lit up” datapath?
b. What should the control signals be set to?
c. Does this instruction use all 5 phases?

1. `add rd rs1 rs2`
2. `beq rs1 rs2 offset`
3. `sw rs2 imm(rs1)`
4. `lw rd imm(rs1)`

1. `add rd rs1 rs2`
2. `beq rs1 rs2 offset`
3. `sw rs2 imm(rs1)`
4. `lw rd imm(rs1)`

- Which wires should hold relevant data?
- What should the control signals be set to?
- Does this instruction use all 5 phases?





1. `add rd rs1 rs2`
2. `beq rs1 rs2 offset`
3. `sw rs2 imm(rs1)`
4. `lw rd imm(rs1)`

- Which wires should hold relevant data?
- What should the control signals be set to?
- Does this instruction use all 5 phases?



Scratch Work:

# Control Logic Truth Table (Partial)

Do Project 3!



| inst[31:0] | BrEq | BrLT | PCSel | ImmSel | BrUn | ASel | BSel | ALUSel | MemRW | RegWEn | WBSel |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ***R-Type*** | * | * | +4 | * | * | Reg | Reg | *(Op)* | Read | 1 | ALU |
| **add** | | | | | | | | Add | | | |
| **sub** | | | | | | | | Sub | | | |
| **addi** | * | * | +4 | I | * | Reg | Imm | Add | Read | 1 | ALU |
| **lw** | * | * | +4 | I | * | Reg | Imm | Add | Read | 1 | Mem |
| **sw** | * | * | +4 | S | * | Reg | Imm | Add | Write | 0 | * |
| **beq** | | | | B | * | PC | Imm | Add | Read | 0 | * |
| **not taken** | 0 | * | +4 | | | | | | | | |
| **taken** | 1 | * | ALU | | | | | | | | |
| **bne** | | | | B | * | PC | Imm | Add | Read | 0 | * |
| **taken** | 0 | * | ALU | | | | | | | | |
| **not taken** | 1 | * | +4 | | | | | | | | |
| **blt taken** | * | 1 | ALU | B | 0 | PC | Imm | Add | Read | 0 | * |
| **bltu taken** | * | 1 | ALU | B | 1 | PC | Imm | Add | Read | 0 | * |
| **jalr** | * | * | ALU | I | * | Reg | Imm | Add | Read | 1 | PC+4 |
| **jal** | * | * | ALU | J | * | PC | Imm | Add | Read | 1 | PC+4 |
| **auipc** | * | * | +4 | U | * | PC | Imm | Add | Read | 1 | ALU |

# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - **Control Logic Design: ROM**
  - Control Logic Design: Combinational Logic
- Instruction Timing

# Two Options for Control Realization

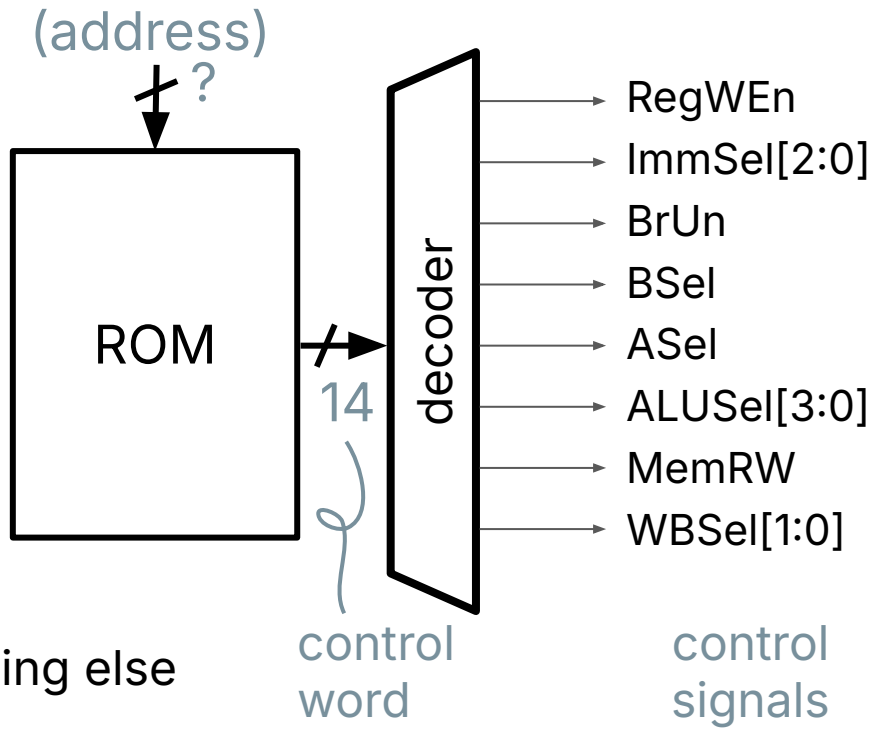


1. Read-Only-Memory (ROM)
    a. Regular structure
    b. Can be easily reprogrammed to…
        i. fix errors
        ii. add instructions
    c. Popular when designing control logic manually
2. Combinational Logic
    a. Instead of ROM, design with gates (AND, OR, NOT)
    b. Popular for chip design (fast)
    c. Today, chip designers use logic synthesis tools to convert truth tables to networks of gates.

# RV32I ROM-based Control



1. Read-Only-Memory (ROM) reads out a word at a given address.
2. To get control signals:
   a. Input Address: Relevant set of instruction bits
   b. Output word: Bits of control signals




1. What is the minimum set of instruction bits needed for input (RV32I)?

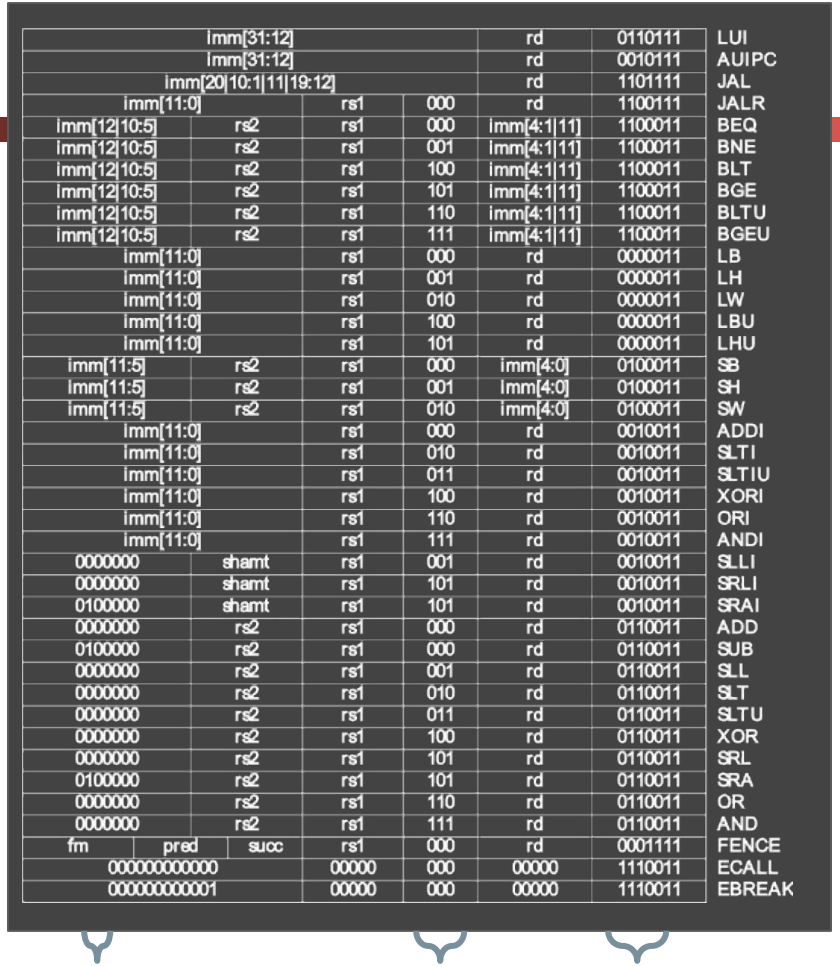
2. Why might PCSel not be known at this time?

**A.** 32
**B.** 17
**C.** 11
**D.** 9
**E.** Something else

# RV32I is a “9-bit ISA”



1. `inst[30]`
   a. `funct7`
   b. `add/sub`, `sll/srl`
2. `inst[14:12]`
   a. `funct3`
3. `inst[6:2]`
   a. `opcode`
   b. inst[`1:0`] fixed in RV32I, changes with extensions (e.g., compressed instructions)

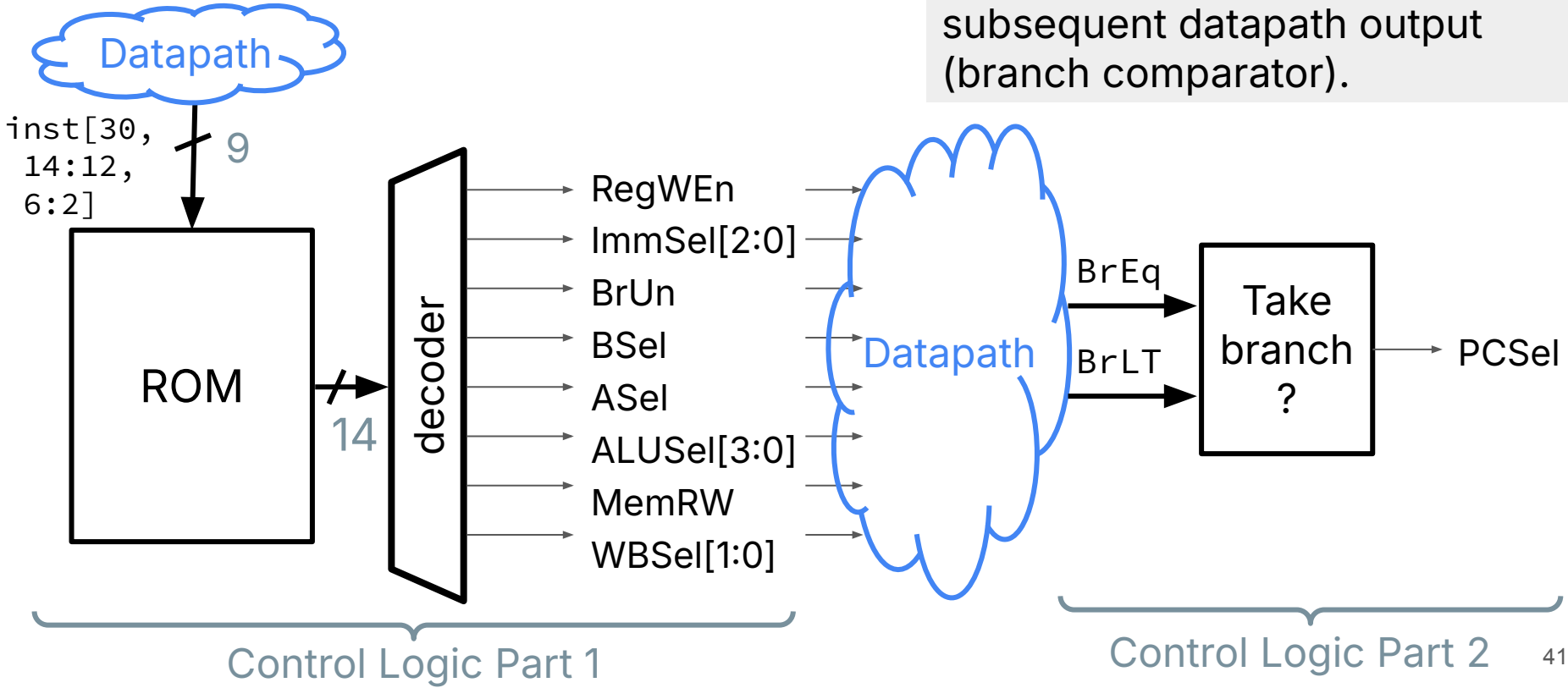


| | | | | | | |
|---|---|---|---|---|---|---|
| imm[31:12] | | | | rd | 0110111 | LUI |
| imm[31:12] | | | | rd | 0010111 | AUIPC |
| imm[20\|10:1\|11\|19:12] | | | | rd | 1101111 | JAL |
| imm[11:0] | | rs1 | 000 | rd | 1100111 | JALR |
| imm[12\|10:5] | rs2 | rs1 | 000 | imm[4:1\|11] | 1100011 | BEQ |
| imm[12\|10:5] | rs2 | rs1 | 001 | imm[4:1\|11] | 1100011 | BNE |
| imm[12\|10:5] | rs2 | rs1 | 100 | imm[4:1\|11] | 1100011 | BLT |
| imm[12\|10:5] | rs2 | rs1 | 101 | imm[4:1\|11] | 1100011 | BGE |
| imm[12\|10:5] | rs2 | rs1 | 110 | imm[4:1\|11] | 1100011 | BLTU |
| imm[12\|10:5] | rs2 | rs1 | 111 | imm[4:1\|11] | 1100011 | BGEU |
| imm[11:0] | | rs1 | 000 | rd | 0000011 | LB |
| imm[11:0] | | rs1 | 001 | rd | 0000011 | LH |
| imm[11:0] | | rs1 | 010 | rd | 0000011 | LW |
| imm[11:0] | | rs1 | 100 | rd | 0000011 | LBU |
| imm[11:0] | | rs1 | 101 | rd | 0000011 | LHU |
| imm[11:5] | rs2 | rs1 | 000 | imm[4:0] | 0100011 | SB |
| imm[11:5] | rs2 | rs1 | 001 | imm[4:0] | 0100011 | SH |
| imm[11:5] | rs2 | rs1 | 010 | imm[4:0] | 0100011 | SW |
| imm[11:0] | | rs1 | 000 | rd | 0010011 | ADDI |
| imm[11:0] | | rs1 | 010 | rd | 0010011 | SLTI |
| imm[11:0] | | rs1 | 011 | rd | 0010011 | SLTIU |
| imm[11:0] | | rs1 | 100 | rd | 0010011 | XORI |
| imm[11:0] | | rs1 | 110 | rd | 0010011 | ORI |
| imm[11:0] | | rs1 | 111 | rd | 0010011 | ANDI |
| 0000000 | shamt | rs1 | 001 | rd | 0010011 | SLLI |
| 0000000 | shamt | rs1 | 101 | rd | 0010011 | SRLI |
| 0100000 | shamt | rs1 | 101 | rd | 0010011 | SRAI |
| 0000000 | rs2 | rs1 | 000 | rd | 0110011 | ADD |
| 0100000 | rs2 | rs1 | 000 | rd | 0110011 | SUB |
| 0000000 | rs2 | rs1 | 001 | rd | 0110011 | SLL |
| 0000000 | rs2 | rs1 | 010 | rd | 0110011 | SLT |
| 0000000 | rs2 | rs1 | 011 | rd | 0110011 | SLTU |
| 0000000 | rs2 | rs1 | 100 | rd | 0110011 | XOR |
| 0000000 | rs2 | rs1 | 101 | rd | 0110011 | SRL |
| 0100000 | rs2 | rs1 | 101 | rd | 0110011 | SRA |
| 0000000 | rs2 | rs1 | 110 | rd | 0110011 | OR |
| 0000000 | rs2 | rs1 | 111 | rd | 0110011 | AND |
| fm \| pred \| succ | | rs1 | 000 | rd | 0001111 | FENCE |
| 000000000000 | | 00000 | 000 | 00000 | 1110011 | ECALL |
| 000000000001 | | 00000 | 000 | 00000 | 1110011 | EBREAK |

# RV32I ROM-based Control



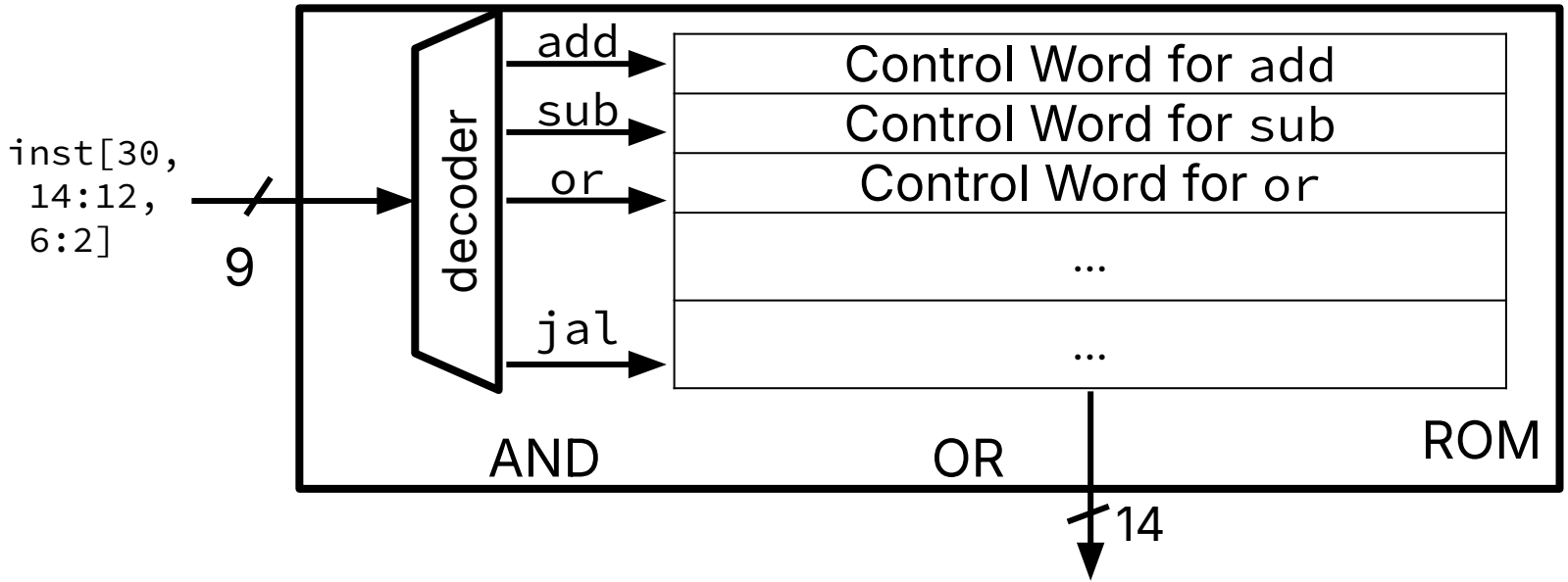
PCSel cannot be encoded in our ROM because it depends on subsequent datapath output (branch comparator).



# ROM Controller Implementation



Under the hood, the ROM controller can be implemented as combinational logic gates. Use Sum of Products on logic table.



# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - **Control Logic Design: Combinational Logic**
- Instruction Timing

# Combinational Logic-based Control Logic



- Much harder than ROM approach
  - We won't be able to help as much in OH
- But also lets you see the beauty of RISC-V's underlying design
- Goal: Build a combinational logic circuit that computes each individual control signal, using the 9 relevant bits of the opcode and funct3/funct7
- Compared to a ROM or comparator-based approach, uses significantly fewer gates (1000+ vs 25 gates)

# [Practice] Combinational Logic-based Control Logic

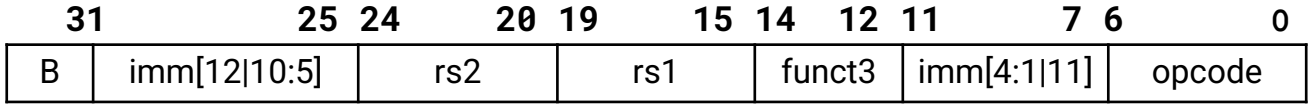


- Write boolean expressions for:
  - BrUn
  - ASel
  - BSel
  - RegWEn
  - You can do more too!

# [Practice] Write a Boolean Expression for BrUn



| Instruction | Opcode | Funct3 |
|---|---|---|
| `beq rs1 rs2 label` | 110 0011 | 000 |
| `bne rs1 rs2 label` | 110 0011 | 001 |
| `blt rs1 rs2 label` | 110 0011 | 100 |
| `bltu rs1 rs2 label` | 110 0011 | 110 |
| `bge rs1 rs2 label` | 110 0011 | 101 |
| `bgeu rs1 rs2 label` | 110 0011 | 111 |



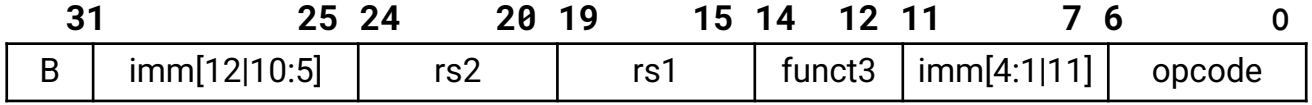

```
BrUn =
!inst[2]&!inst[3]&!inst[4]&inst[5]&inst[6]&inst[13]
```

# [Practice] Write a Boolean Expression for BrUn



| Instruction | Opcode | Funct3 |
|---|---|---|
| `beq rs1 rs2 label` | `110 0011` | `000` |
| `bne rs1 rs2 label` | `110 0011` | `001` |
| `blt rs1 rs2 label` | `110 0011` | `100` |
| `bltu rs1 rs2 label` | `110 0011` | `110` |
| `bge rs1 rs2 label` | `110 0011` | `101` |
| `bgeu rs1 rs2 label` | `110 0011` | `111` |

For non-Branch instructions, BrUn is *, not 0. So we don't actually need to check the opcode bits!




```
BrUn = inst[13]
```

# Combinational Logic-based Control Logic



- Much harder than ROM approach
  - We won't be able to help as much in OH
- But also lets you see the beauty of RISC-V's underlying design
- Goal: Build a combinational logic circuit that computes each individual control signal, using the 9 relevant bits of the opcode and funct3/funct7
- Compared to a ROM or comparator-based approach, uses significantly fewer gates (1000+ vs 25 gates)
- General approach: Find patterns in instructions, take advantage of * values to minimize circuit complexity. Handle each bit separately.

# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- **Instruction Timing**

# Instructions' Critical Path



For each of the below instructions, what is the delay along the critical path?

1. **`add rd rs1 rs2`**
2. `lw rd imm(rs1)`

**A.** tclk_q + tAdd + tIMEM + tReg + tBrComp + tALU + tDMEM + tmux + tSetup

**B.** tclk_q + tIMEM + tReg + tmux + tALU + tmux + tsetup

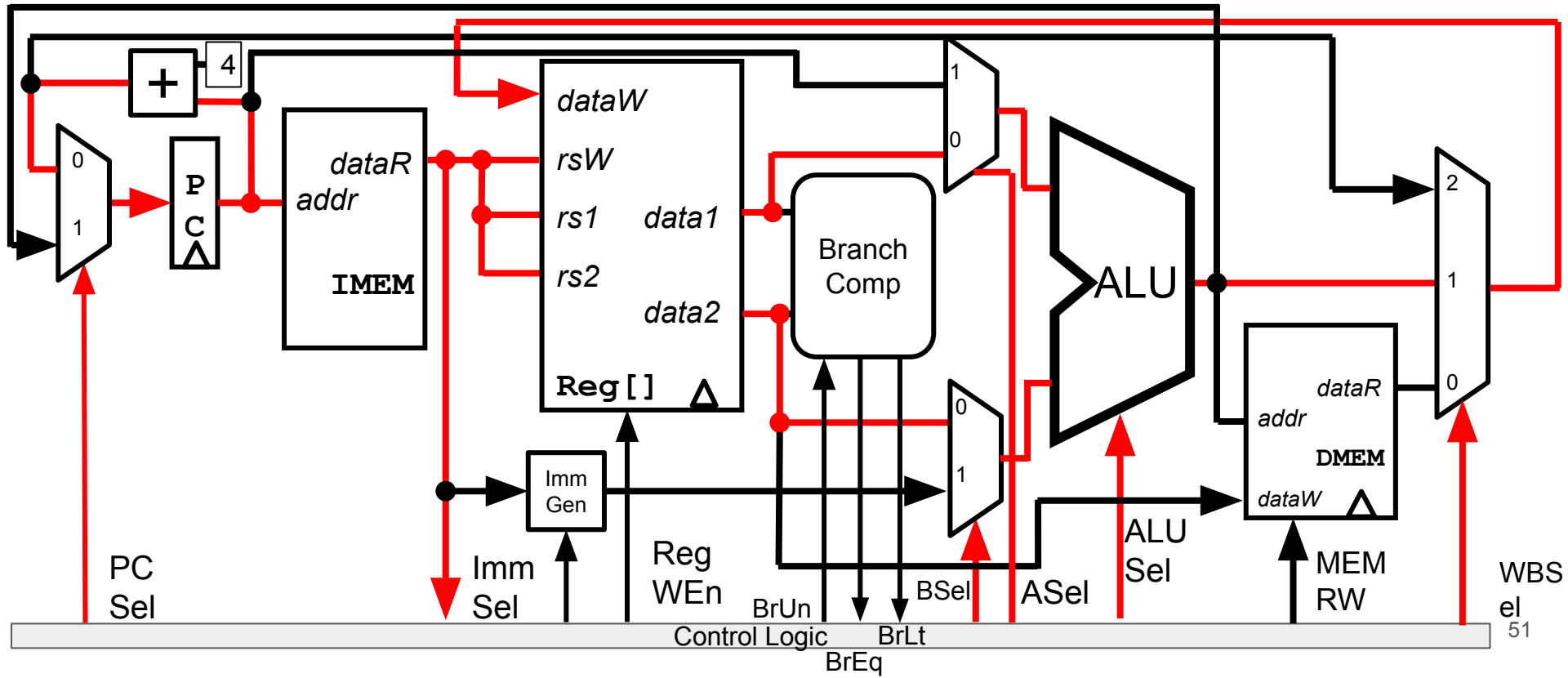
**C.** tclk_q + tIMEM + max{tReg, tImm} + tALU + 2*tmux + tDMEM + tSetup

**D.** tclk_q + tIMEM + max{tReg, tImm} + tALU + 3*tmux + tSetup

**E.** Something else

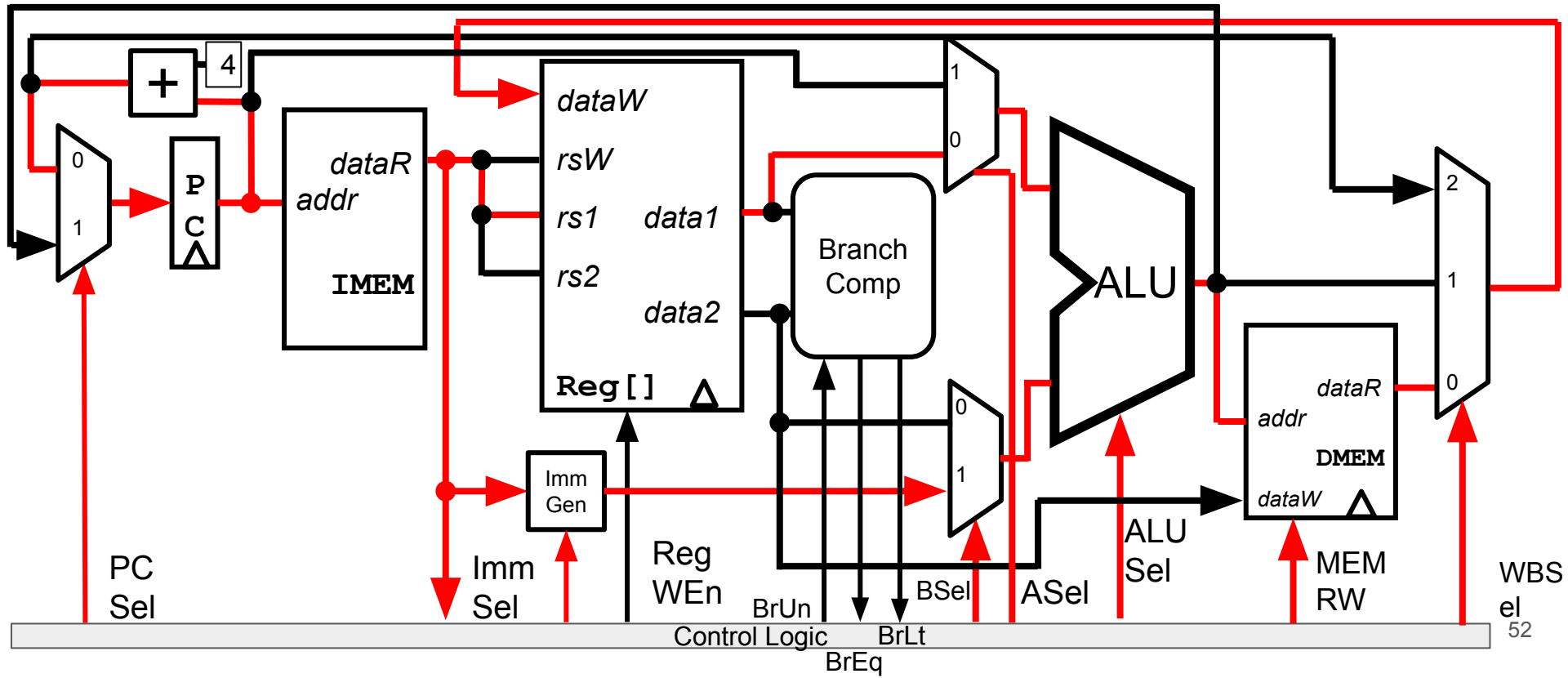
# Critical Path for Add: tclk_q + tIMEM + tReg + tmux + tALU + tmux + tsetup





# Critical Path for lw: `tclk_q + tIMEM + max{tReg, tImm} + tALU + 2*tmux + tDMEM + tSetup`

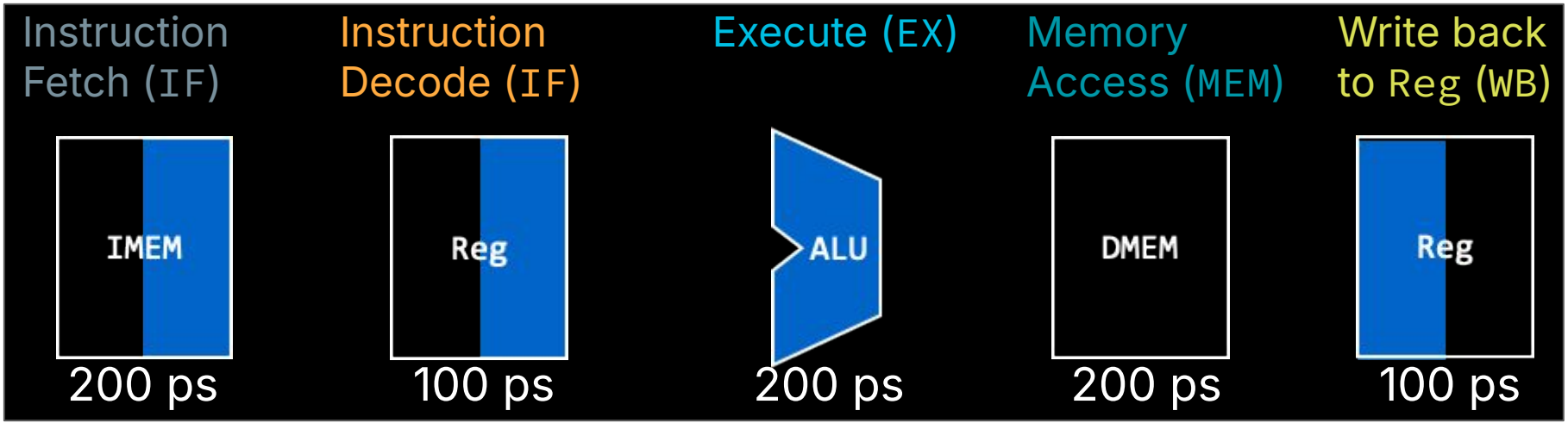




# Instruction Timing, Divided by 5 Phases



- To estimate maximum clock frequency, analyze at a higher level.
- Assume each phase is **dominated** by major functional HW units:



# Compute Clock Period



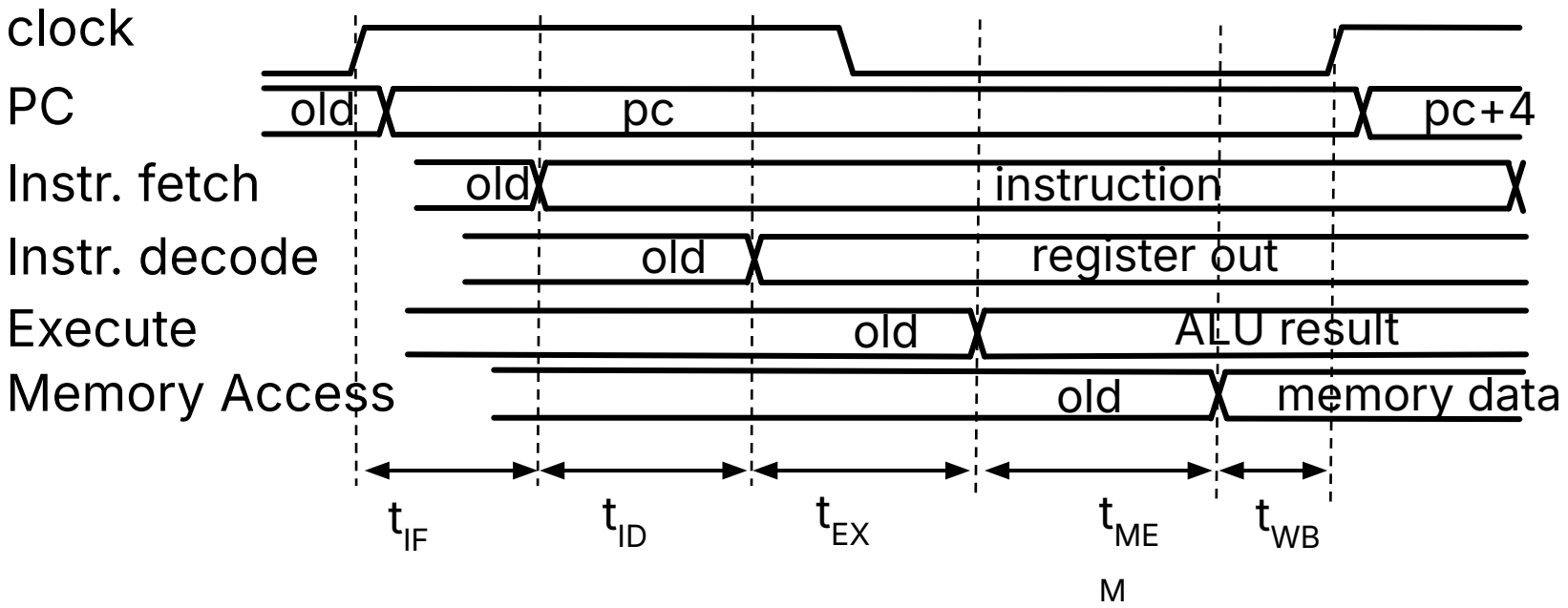
- The clock cycle must encompass the **longest critical path delay** incurred by any instruction.

| Instruction | IF (200ps) | ID (100ps) | EX (200ps) | MEM (200ps) | WB (100ps) | Total |
|---|---|---|---|---|---|---|
| `add` | X | X | X | | X | 600ps |
| `beq` | X | X | X | | | 500ps |
| `jal` | X | X | X | | | 500ps |
| `lw` | X | X | X | X | X | 800ps |
| `sw` | X | X | X | X | | 700ps |

  - Maximum Clock Frequency:
    - $f_{max}$ = 1/800 ps = 1.25 GHz
    - However, not all instructions use all HW blocks.
  - Many blocks are idle most of the time…!
    - Conversely, each phase just takes 200 ps → could clock 5GHz…??

# 5-Phase Timing, In Detail





# Agenda



- Implementing Branches
- Implementing U-types
- Implementing ImmGen
- Datapath and Control
  - Control Logic Design: ROM
  - Control Logic Design: Combinational Logic
- Instruction Timing
- **And in Conclusion**

# Call home, we've made HW/SW contact!



High Level Language Program (e.g., C)

*Compiler*

Assembly Language Program (e.g., RISC-V)

*Assembler*

Machine Language Program (RISC-V)

Hardware Architecture Description (e.g., block diagrams)

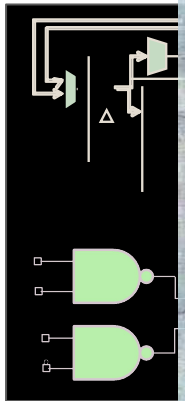
*Architecture Implementation*

Logic Circuit Description (Circuit Schematic Diagrams)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw      x3, 0(x10)
lw      x4, 4(x10)
sw      x4, 0(x10)
sw      x3, 4(x10)
```

```
1000 1101 1110 0010
1000 1110 0001 0000
1010 1110 0001 0010
1010 1101 1110 0010
```



# “And in conclusion…”



- We have built a processor!
  - Capable of executing all RISC-V instructions in one cycle each
- Not all HW units used by all instructions
  - The **critical path changes** based on instruction.
- 5 Phases of execution
  - IF, ID, EX, MEM, WB
  - Not all instructions are active in all phases
- Controller specifies how to execute instructions
  - Implemented as ROM or logic
- Next:
  - Making our single-stage CPU faster